
***Automated AQI Prediction System
Gujrat, Pakistan – 72-Hour Forecast***

***Submitted by:
Muhammad Arhum***

Data Science Intern

***Program:
10P Shine Internship Program***



Duration:

December 2025- February 2026

Table of Contents

1. Introduction	3
2. Data Collection.....	3
3. MongoDB as Feature Store.....	4
4. AQI Calculation	4
5. Feature Engineering	5
6. Model Training.....	6
7. Feature Importance.....	7
8. Pipelines	a8
9. Web Application Dashboard.....	9
10. Key Challenges and Solutions	9

AQI Prediction System – Gujrat, Pakistan

1.Introduction

Air pollution poses a major environmental concern in Pakistan. Reliable AQI forecasts allow people to plan outdoor activities safely and support policymakers in taking prompt actions. This project builds an automated machine learning system capable of predicting AQI levels in Gujrat up to 72 hours ahead.

Objectives

- Develop a machine learning model for AQI prediction achieving an R^2 of at least 0.75.
- Set up automated hourly data collection from relevant sources.
- Establish a production pipeline for daily model training and forecasting.
- Deploy an interactive dashboard to visualize predictions and trends.

Scope

- **Location:** Gujrat, Pakistan (LATITUDE=32.5731, LONGITUDE=74.1005)
- **Historical data collection:** 3 months
- **AQI calculation:** Using EPA standards
- **Feature engineering:** 29 features
- **Models:** Random Forest, XGBoost, LightGBM
- **Deployment:** Automated (GitHub Actions)
- **Integration:** MongoDB and Streamlit dashboard

2.Data Collection

Table 1.1. APIs and its Selection

API	Access	Historical Data	Selected
AQICN	DNS blocked, VPN needed	2 months	No
OpenWeather	DNS blocked, VPN needed	2 months	No

OpenMeteo	No restrictions	90+ days	Yes
-----------	-----------------	----------	-----

Reason for selecting OpenMeteo: free access, no API key requirement, no DNS blocking, and sufficient historical data.

3. MongoDB as Feature Store

Raw weather and air quality data are collected hourly through the Open-Meteo API and stored in MongoDB, which serves as a centralized feature store. This ensures that historical and real-time data are readily accessible for processing, feature engineering, and model training.

Data is fetched hourly using Python scripts, requesting weather variables (temperature, humidity, wind speed, precipitation, cloud cover) and pollutant levels. Responses are parsed, cleaned, structured into tables, and inserted into MongoDB with timestamps and location metadata. This setup enables reproducible feature engineering and automated model training.

4. AQI Calculation

AQI for a single pollutant

For a pollutant with measured concentration $C_{measured}$, AQI is calculated using EPA's piecewise linear function:

$$AQI_{pollutant} = \frac{I_{high} - I_{low}}{C_{high} - C_{low}} \cdot (C_{measured} - C_{low}) + I_{low}$$

Where:

- C_{low}, C_{high} = pollutant concentration breakpoints
- I_{low}, I_{high} = corresponding AQI breakpoints (e.g., PM2.5: 0–12 $\mu\text{g}/\text{m}^3 \rightarrow$ AQI 0–50, 12.1–35.4 $\mu\text{g}/\text{m}^3 \rightarrow$ AQI 51–100, etc.)

Overall AQI

$$AQI_{overall} = \max(AQI_{PM2.5}, AQI_{PM10}, AQI_{O3}, AQI_{NO2}, AQI_{SO2}, AQI_{CO})$$

AQI Category

Table 1.2. AQI range and its Category

AQI Range	Category
0–50	Good
51–100	Moderate
101–150	Unhealthy for Sensitive Groups
151–200	Unhealthy
201–300	Very Unhealthy
>300	Hazardous

5.Feature Engineering

After calculating AQI and storing the data, a reduced feature engineering pipeline generates 29 features to capture temporal patterns and relationships.

Table 1.3. Selected Features with categories

Feature Category	Features
Pollutants (6)	pm10, pm2_5, carbon_monoxide, nitrogen_dioxide, sulphur_dioxide, ozone
Weather (5)	temperature_2m, relative_humidity_2m, wind_speed_10m, precipitation, cloud_cover
Time (6)	hour_sin, hour_cos, dow_sin, dow_cos, month_sin, month_cos
Lag (4)	aqi_lag_1, aqi_lag_3, aqi_lag_6, aqi_lag_24
Rolling (4)	aqi_roll_mean_6, aqi_roll_std_6, aqi_roll_mean_24, aqi_roll_std_24
Momentum (1)	aqi_change_1h
Interaction (3)	temp_humidity, wind_x, wind_y

Cleaned datasets are saved back into MongoDB for modeling.

6. Model Training

Process

- Source: engineered_data_final in MongoDB
- Models trained: Random Forest, XGBoost, LightGBM
- Split: 80% training, 20% testing
- Input: Pollutants, weather, time, lag, rolling, momentum, interaction features
- Output: AQI

Hyperparameters:

- Random Forest: 150 trees, max depth 8, min samples per leaf 8
- XGBoost: depth 3, learning rate 0.02, L1/L2 regularization
- LightGBM: 120 iterations, num_leaves=7, learning rate 0.02, subsample 0.7

Models are evaluated using R^2 , RMSE, and MAE. The best model is serialized and stored in MongoDB with metadata.

Model Performance:

Table 1.4. Model Comparison Results

Model	R^2 Score	MAE	RMSE
Random Forest	0.775	11.65	23.32
XGBoost	0.698	17.20	27.03
LightGBM	0.718	16.86	26.11

Best Model: Random Forest

- Highest R^2 (0.775)
- Lowest RMSE (23.32)
- Lowest MAE (11.65)
- Efficient training and moderate memory footprint

Reason of Selecting Model:

This performance is suitable for production because:

- **Inherent Uncertainty:** AQI is affected by many stochastic factors such as traffic patterns, weather changes, and emission fluctuations.
- **Proper Validation:** Time-series split ensures no future data leaks into training.
- **Past Data Only:** Predictions are based solely on historical observations.
- **Honest Metrics:** Moderate R^2 indicates models are not overfitted; extremely high R^2 (>0.95) often signals data leakage or unrealistic performance.

7.Feature Importance

Table 1.5. Features Importance using SHAP

Rank	Feature	Mean Absolute SHAP
1	aqi_lag_1	10.24
2	aqi_roll_mean_6	5.63
3	ozone	5.11
4	aqi_lag_24	4.93
5	pm2_5	4.38
6	aqi_change_1h	4.04
7	pm10	4.01
8	sulphur_dioxide	2.45
9	aqi_roll_mean_24	2.28
10	nitrogen_dioxide	2.00

Key Insights:

- Recent AQI (lagged) dominates predictions
- Primary pollutants (ozone, pm2.5, pm10, SO2, NO2) are critical
- Momentum and rolling features enhance short-term prediction accuracy

8. Pipelines

Hourly Pipeline

- Collects, processes, and stores AQI/weather data hourly
- Converts timestamps to PKT
- Computes AQI and engineered features
- Stores processed features in `engineered_data_final` collection

Daily Model Training & Registration

- Daily retraining using latest engineered features
- Three models trained (RF, XGBoost, LightGBM)
- Best model selected based on R^2
- Metadata stored in `final_model_registry` for versioning

72-Hour Prediction Pipeline

- Loads latest best model from MongoDB
- Sequentially generates 72-hour forecast features
- Updates lag, rolling, and interaction features dynamically
- Stores predicted AQI in MongoDB with timestamps and metadata

GitHub Actions & Automated CI/CD Pipeline

- Hourly Feature Pipeline: Automatically collects and processes the latest weather and pollutant data every hour
- Daily Model Training Pipeline: Trains and evaluates models daily, selecting the best-performing model for production
- CI/CD Automation:
 - Installs dependencies and sets up the environment
 - Runs pipelines (feature engineering, training, prediction)
 - Deploys updated Streamlit dashboard to the cloud

- Monitoring & Logging: Captures errors, retries failed steps, and maintains deployment logs for traceability
- Benefits: Ensures real-time AQI predictions, continuous model improvement, and a reliable production-ready system

9. Web Application Dashboard

- Streamlit-based interface
- WHO Dark Mode + Glass UI
- Features:
 - Animated AQI gauge
 - 72-hour forecast interactive line chart
 - Model evaluation panel (R^2 comparison, selected model)
 - Automated cloud-ready updates

10. Key Challenges and Solutions

Table 1.6. Challenges faced During Project and its solutions

Challenge	Solution
DNS blocking of AQICN/OpenWeather	Evaluated multiple APIs, selected OpenMeteo
Limited historical data (last 3 months)	Continuous collection strategy, 90+ days initial backfill
Proper time-series validation	Implemented temporal split, no shuffling
Feature engineering complexity (lags, rolling, interaction)	Automated sequential feature pipelines, tested feature consistency
Model selection and performance tracking	Trained multiple models (RF, XGBoost, LightGBM), registered best model in MongoDB with metadata
Prediction consistency for 72-hour forecast	Sequential update of lag & rolling features during prediction

Production reliability	Error handling, retry logic, logging, and monitoring for pipelines and dashboard
Dashboard responsiveness & UX	WHO Dark Mode + Glass UI, interactive Plotly charts, mobile-friendly layout